

MODULE 5.1 · Concept 4 of 7

ReAct Loop Visualization

Interactive visual walkthrough — seeing where agent behaviour unfolds and where it breaks

7 min

Duration

B2 – Understand

Bloom's

L2 – Intermediate

Level

TH-000003

Prerequisite

AI-AG-IN-VZ-000001

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — THE TRACE UNFOLDING

Visualizing Agent Behavior Over Time



Building intuition for how Thought→Action→Observation cycles work

Why Visualization Matters

Seeing a trace is worth a thousand descriptions

Agents are hard to understand from text alone because:

- The loop is dynamic — each step depends on the previous step's result
- Multiple things happen in sequence — it's easy to lose track
- Failures are subtle — a wrong tool call looks the same as a correct one in raw text
- Context grows over time — you need to see token accumulation

Visualization makes the implicit structure explicit. You can SEE the branching, looping, and decision points.

Visual Trace: Steps 1-2 (Query → Thought → Action)

The agent receives the query and plans its first action

USER QUERY

input

"What was the GDP growth rate of India last quarter?"

THOUGHT 1

generated

"I need to search for India's most recent GDP growth data. Let me search for Q3 2025 GDP."

ACTION 1

generated

`web_search(query="India GDP growth rate Q3 2025")`

← *At this point, the orchestrator sends the search query to the real API. The agent waits for the result.*

Visual Trace: Steps 3-4 (Observation → Reasoning)

Real data enters the context — the agent reasons about it

OBSERVATION 1

injected (real data)

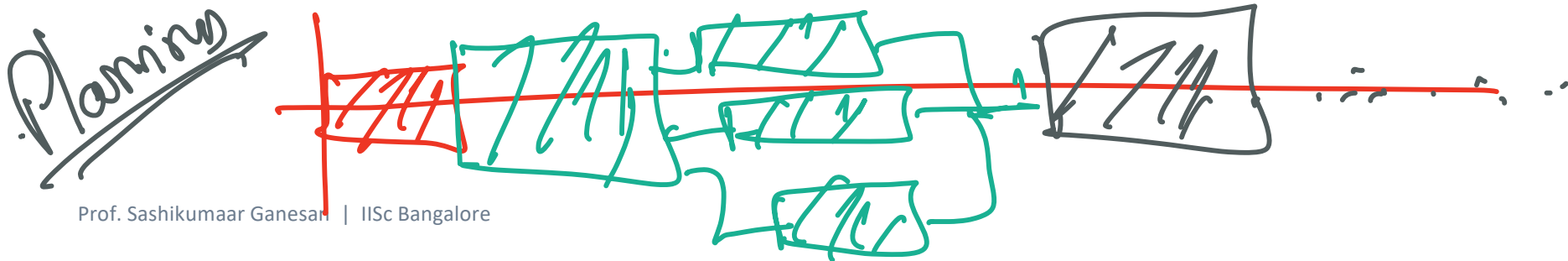
"India's GDP grew 6.7% in Q3 FY2025 (July-Sept 2025), according to the Ministry of Statistics."

THOUGHT 2

generated

"I found the growth rate: 6.7% for Q3 FY2025. The user asked for the last quarter, and this is the most recent data. I have enough to answer."

The Observation (blue) is the ONLY step that brings real-world data. Everything else is model-generated.



Visual Trace: Steps 5-6 (Answer — Loop Exits)

The agent has enough information and delivers the final answer

ANSWER

"India's GDP grew at 6.7% in the most recent quarter (Q3 FY2025, July–September 2025), according to official data from the Ministry of Statistics."

Total trace: 6 steps, 1 tool call, ~700 tokens, ~\$0.0004

Notice what made this trace efficient:

- Specific search query ('India GDP growth rate Q3 2025') → got the exact data needed in one search
- The Observation contained everything needed — no second search required
- Thought 2 correctly assessed: 'I have enough to answer' → loop exits
- Answer cites the source (Ministry of Statistics) — grounded in the Observation

Full Trace as Timeline

All steps on one view — the complete agent execution



Token accumulation →

500 tok

550 tok

600 tok

750 tok

810 tok

920 tok

Legend:

■ T = Thought

■ A = Action

■ O = Observation

■ ANS = Answer

SECTION 2 — WHERE THINGS GO WRONG

Three Common Failure Patterns

Recognizing failures in traces is the key debugging skill

Failure 1: Hallucinated Tool Call

The agent calls a tool that doesn't exist

Thought: "I need to check the stock price. Let me use the financial API."

Action: `stock_price_api(ticker="RELIANCE")`

Observation: "Error: tool 'stock_price_api' not found. Available tools: web_search, calculator"

The agent invented 'stock_price_api' — it doesn't exist in the tool registry.

Fix: Return clear error listing available tools. The agent will self-correct in the next Thought and use web_search instead.

Failure 2: Infinite Loop

The agent keeps searching without converging to an answer

Thought 1: "Let me search for India GDP growth rate"

Action: `web_search("India GDP growth")`

Obs: "6.7% in Q3 2025"

Thought 2: "I should verify this with another source"

Action: `web_search("India GDP Q3 2025 official")`

Obs: "6.7% according to NSO"

Thought 3: "Let me check if there's a revised figure..."

Action: `web_search("India GDP revised Q3 2025")`

... continues for 15+ iterations ...

Fix: Set `max_iterations=5`. Add convergence check: if same data appears twice, stop.

Failure 3: Wrong Tool Selection

Agent uses calculator when it needs search — or vice versa

User: "What is Apple's current stock price?"

Thought: "I need to calculate Apple's stock price..."

Action: calculator("AAPL")

Observation: "Error: invalid mathematical expression 'AAPL'"

Root cause: Ambiguous tool descriptions. 'calculator' description didn't specify it's for MATH ONLY.

Fix: Add negative examples to tool descriptions: 'Do NOT use calculator for lookups, prices, or non-math queries.'

Context Window Token Distribution

Color-coded view: where tokens go in a typical 5-iteration trace



Observations are the largest variable cost — they come from external tools and can be very long.

Optimization strategies:

- Truncate long tool results to relevant sections
- Summarize multi-page search results before injecting
- Keep system prompt concise — it's a fixed cost on every call
- Reduce tool descriptions to minimum needed for correct selection

Debugging Heuristic: Read Traces Backwards

Start from the failure, trace back to the root cause

- | | | | |
|---|----------------------------------|---|---|
| 1 | Read the final Answer | Is it correct? Does it match the user's question? | If wrong → go to step 2 |
| 2 | Read the last Observation | Did the tool return correct/relevant data? | If bad data → bad Action (step 3) |
| 3 | Read the last Action | Was the right tool called with correct params? | If wrong tool → bad Thought (step 4) |
| 4 | Read the last Thought | Does the reasoning logically lead to this Action? | If illogical → prompt engineering issue |

Key Takeaways

The essential points from this concept

- Visualization makes agent behavior tangible** — see the branching, looping, and decision points.
- Three failure patterns to recognize:** hallucinated tools, infinite loops, wrong tool selection.
- Observations (blue) are the only real-world data** — everything else is model-generated.
- Debug backwards:** Answer → Observation → Action → Thought. Find where the trace went wrong.
- Context token distribution matters:** Observations are 25% of tokens. Truncate long results.

Next: AI-AG-IN-TH-000004 — When to Reason vs. Act

AI-AG-IN-VZ-000001 Complete

ReAct Loop Visualization

Concept 4 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi